Enrichment Lecture: Memory-Conscious Models of Computation

Design and Analysis of Algorithms

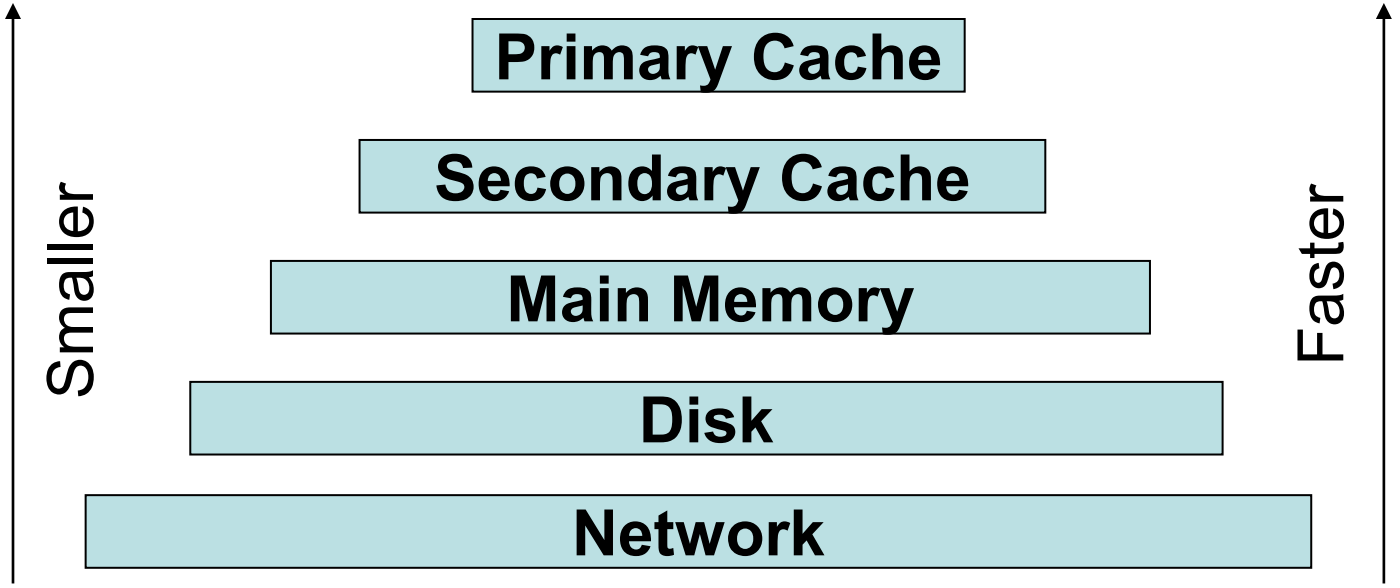
Brian C. Dean

School of Computing

Clemson University

Hierarchical Memory Layout

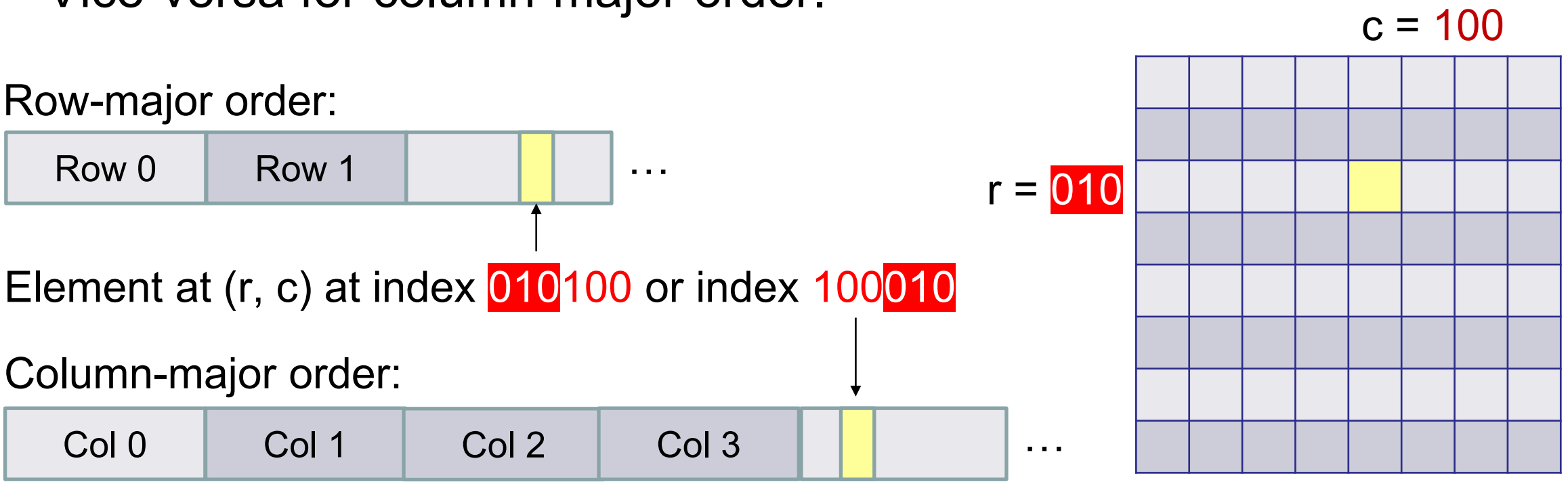
- The RAM model assumes every memory access takes O(1) time.
- Reality is more complicated. Most memories have a hierarchical structure, with block transfers between levels.



Memory segmented into blocks of size B

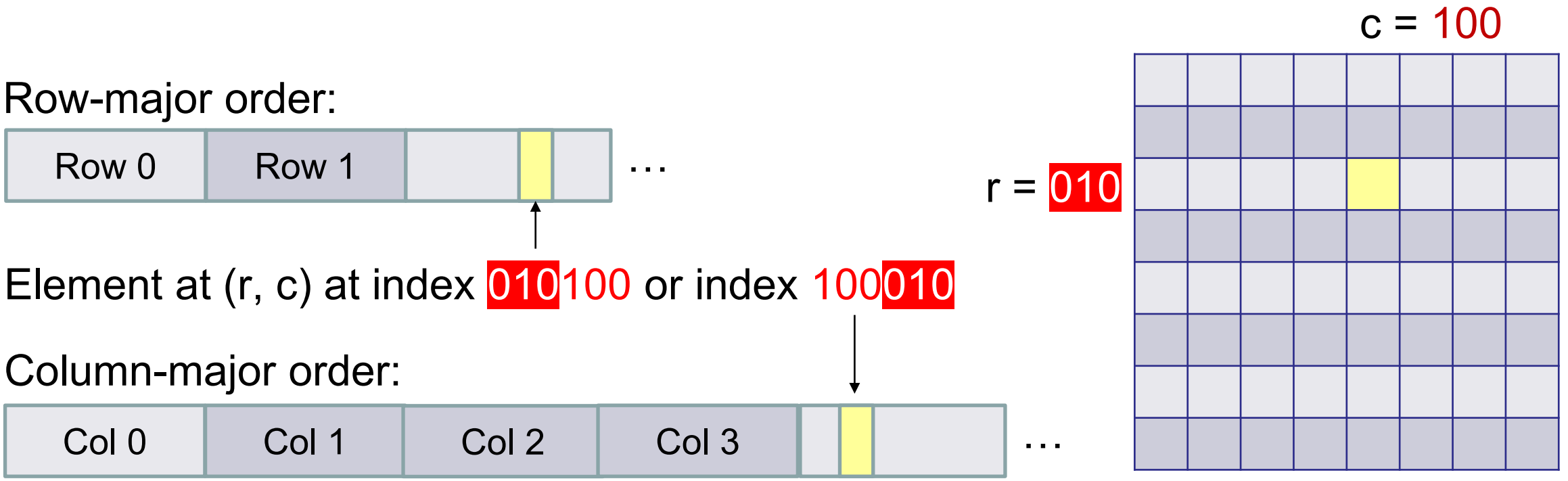
Cache-Friendly Memory Access Patterns

- Storing a matrix in row-major order might work well for access patterns that scan across rows, but not for scanning down columns.
- Vice-versa for column-major order.



Cache-Friendly Memory Access Patterns

- Store element at (r, c) at index interleave(r, c) = 011000.
- Small changes to r or c cause less dramatic changes in index.

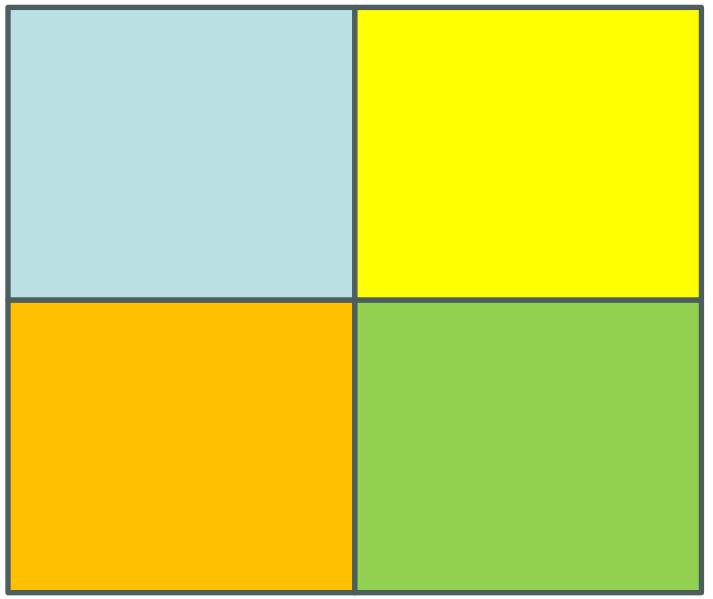


Cache-Friendly Memory Access Patterns

- Store element at (r, c) at index interleave(r, c) = 011000.
- Small changes to r or c cause less dramatic changes in index.

Quadrant order in memory:

Recursive structure same as a “quadtree” data structure for storing 2D data.

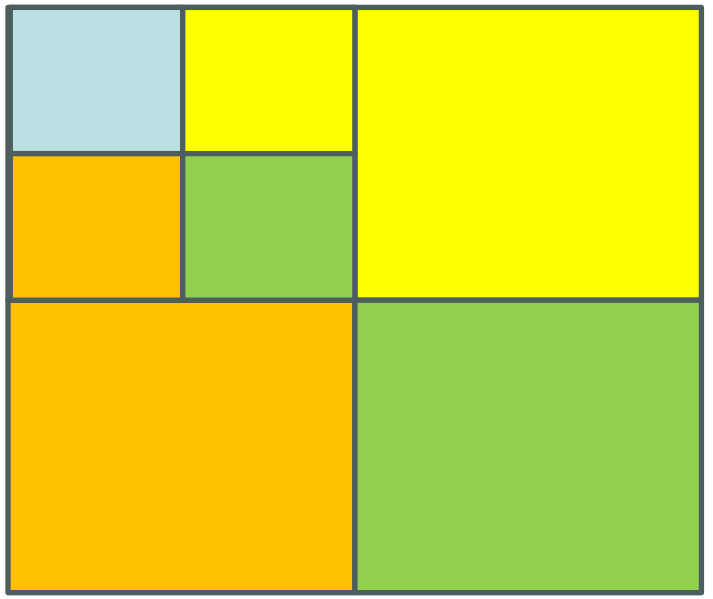


Cache-Friendly Memory Access Patterns

- Store element at (r, c) at index interleave(r, c) = 011000.
- Small changes to r or c cause less dramatic changes in index.

Quadrant order in memory:

Recursive structure same as a “quadtree” data structure for storing 2D data.

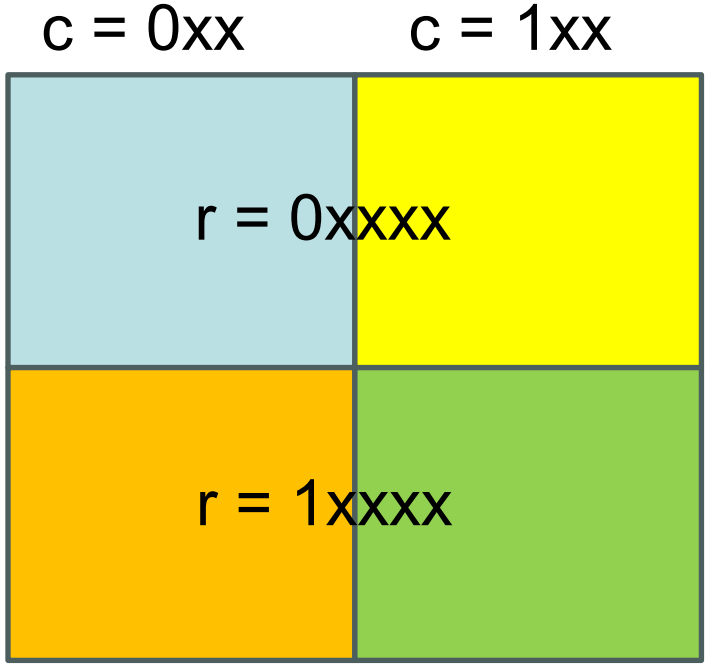


Cache-Friendly Memory Access Patterns

- Store element at (r, c) at index interleave(r, c) = 011000.
- Small changes to r or c cause less dramatic changes in index.

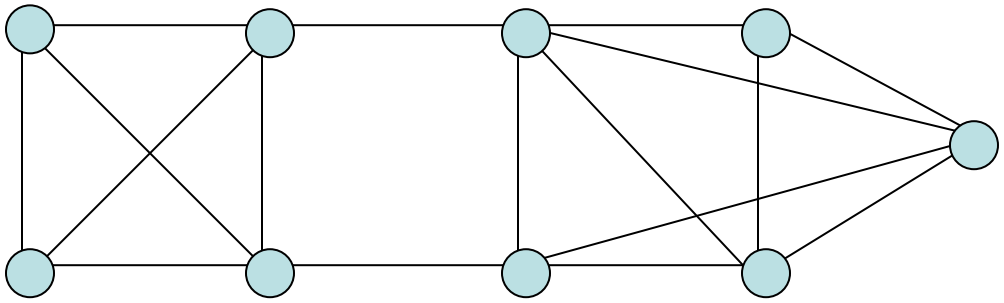
Quadrant order in memory:

Recursive structure same as a “quadtree” data structure for storing 2D data.



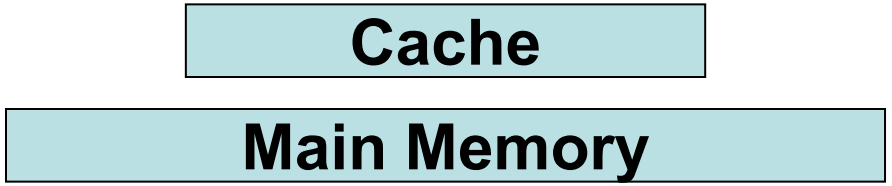
Cache-Friendly Memory Access Patterns: Graph-Structured Data

- For graph / network structured data, how might you serialize the nodes in memory so that neighbors tend to be nearby?
- This can also help with visualization, especially if you want a good 2D instead of a 1D embedding of your network.



A Model of Computation Based on a Simple 2-Level Memory Model

- Two layers of memory:
  - Fast cache of size M segmented into M / B blocks of size B.
  - Slower main memory, also segmented into blocks of size B.



- If we access an element not in the cache, we transfer the entire block (of size B) containing the element into the cache.
- Running time = total # of block transfers, and that's it!

  (Neglect running time of all other operations like addition, etc., since these happen in CPU registers and typically run much faster than memory transfers).

Cache Memory Assumptions – The Fine Print…

- Cache contains M / B blocks each of size B.
- For simplicity, assume cache…
  - is **fully associative** (so each memory block can reside anywhere in cache).
  - uses an **optimal / ideal** page replacement policy (so next block evicted is the one to be used farthest ahead in the future – even though unrealistic).
- If algorithm running time is $T(M, B) = O(T(M / 2, B))$ on an ideal cache (i.e., performance slows by a constant factor when the cache size is halved), then its running time will be $\Theta(T(M, B))$ using either:
  - **LRU** (least recently used) page replacement, or
  - **FIFO** (first-in-first-out) page replacement.

  (both of these policies are common in practice)

Cache-Aware Algorithms

- If an algorithm or data structure knows M and B, it can optimize its performance accordingly to minimize # of block transfers.

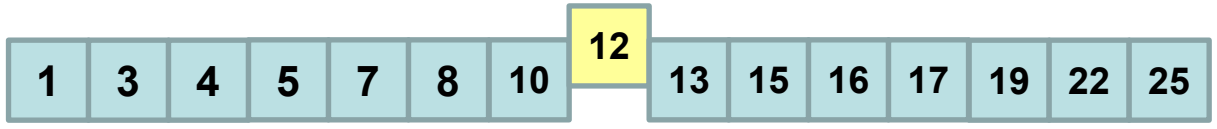
- **Example:** Searching a sorted length-n array.
- Optimal number of block transfers: $O(\log_B n)$.
- Regular binary search does not achieve this, but a generalization with branching factor tuned to B does work…

Binary Search Isn't Optimal…

- Starting with sorted array of size n.
- Problem size effectively halved at each step.
- Once we narrow to a problem of size < B that lies within a block (O(log (n/B)) steps) all remaining operations are "free".
- However, log (n/B) = log n – log B, whereas

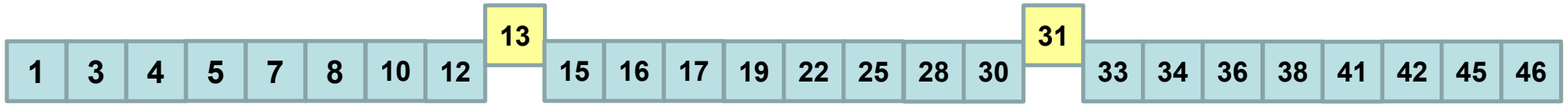
  $O(\log_B n) = O(\log n / \log B)$ is optimal.

Memory segmented into size-B blocks



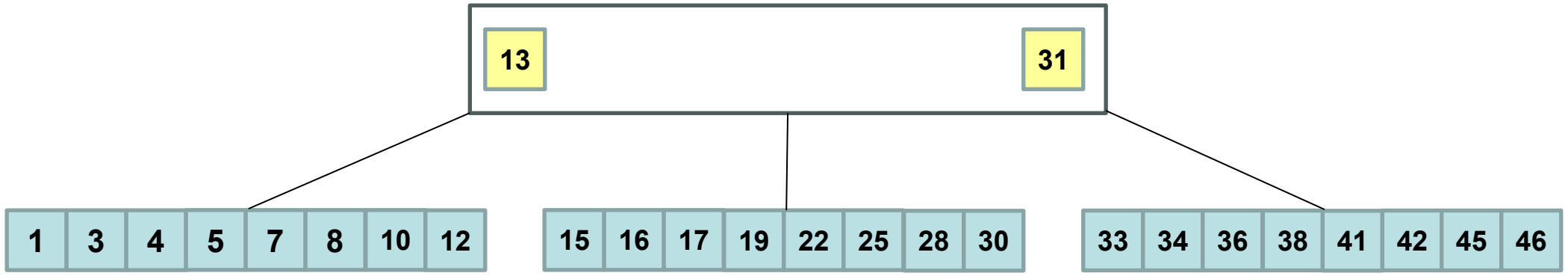
Generalizing Binary Search

- Suppose B = 2 (two elements fit in a block)



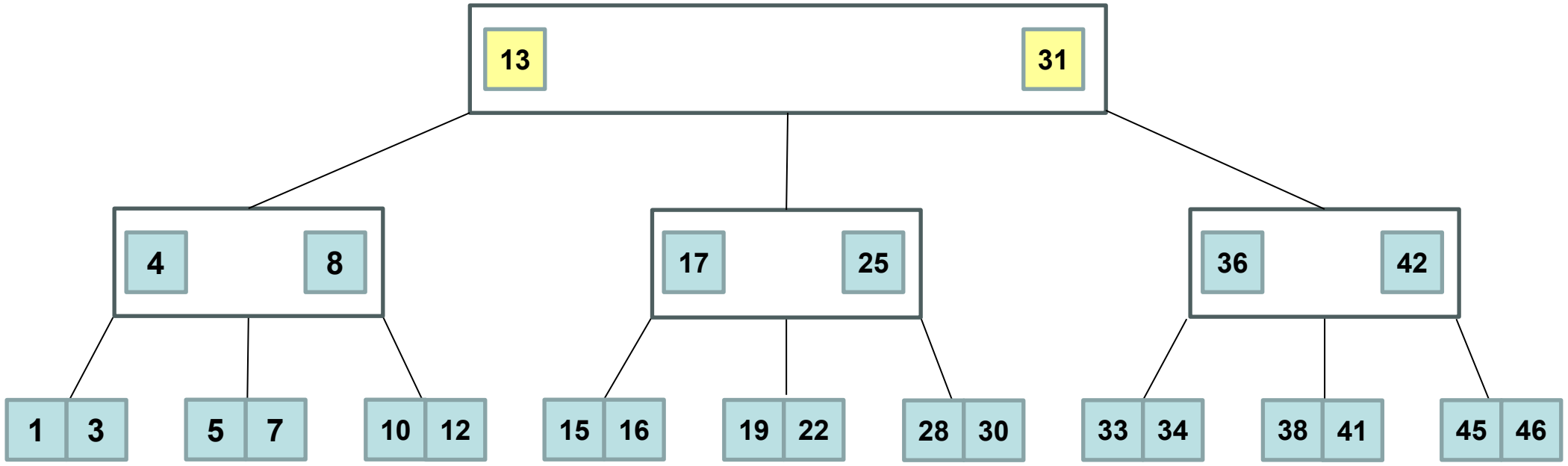
Generalizing Binary Search

- Each step divides our problem now into B+1 pieces (versus just 2 pieces for binary search).



Generalizing Binary Search

- The resulting structure is called a B-tree, and it takes $O(\log_B n)$ steps (optimal) for searching.



Cache-Oblivious Algorithms

- We often don't know M and B, and in a multi-level memory system these parameters may vary substantially from level to level…
- An algorithm is said to be **cache-oblivious** if it doesn't know M or B, and yet its running time is always within a constant factor of an optimal cache-aware algorithm.
- **Example:** Reversing a length-n array.
  - Algorithm: scan and swap from both ends inward.
  - This uses n / B block transfers, which is optimal.
- On a multi-level memory with O(1) levels, since the running time of a cache-oblivious algorithm is within O(1) of optimal between each successive pair of levels, it will be within O(1) of optimal overall!

Cache-Oblivious Algorithms

- Simple “textbook” algorithms are often not cache-oblivious…
- Examples:
  - **Searching a sorted array with binary search**. Runs in O(log n – log B) time versus optimal $O(\log_B n)$.
  - **Sorting**. Merge sort runs in O((n/B) log (n/B)) time, versus optimal $O((n/B) \log_{M/B} (n/B))$.
- For many algorithm and data structure problems, it is interesting (and sometimes quite challenging) to try and develop cache-oblivious solutions!

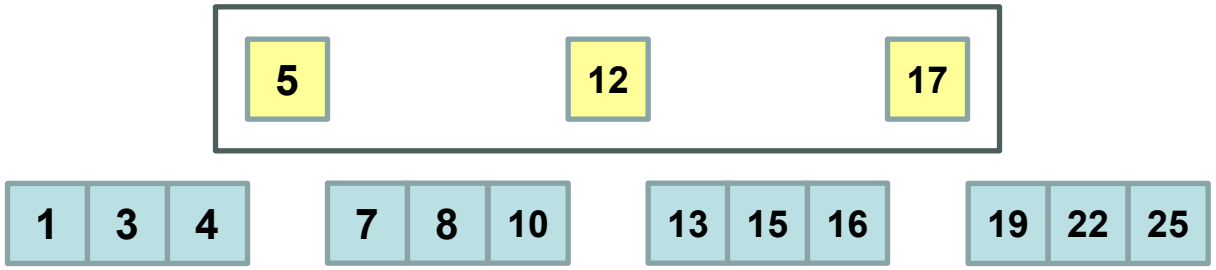
Cache-Oblivious Searching

- Consider the problem of searching a sorted length-n array in the comparison model.
- Optimal cache-aware running time: $O(\log_B n)$ using a B-tree.

Cache-Oblivious Searching

- Consider the problem of searching a sorted length-n array in the comparison model.
- Optimal cache-aware running time: $O(\log_B n)$ using a B-tree.
- Divide into $\sqrt{n}$ blocks of size $\sqrt{n}$. This turns our original search problem of size n into…

  2 problems of size $\sqrt{n} = n^{1/2}$



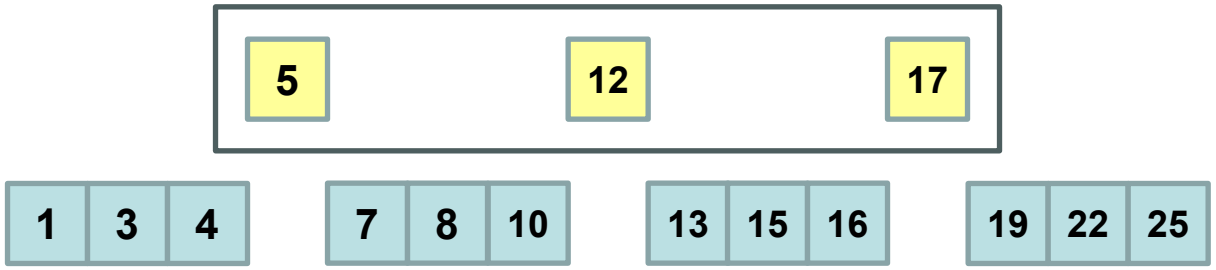
Cache-Oblivious Searching

- Consider the problem of searching a sorted length-n array in the comparison model.
- Optimal cache-aware running time: $O(\log_B n)$ using a B-tree.
- Divide into $\sqrt{n}$ blocks of size $\sqrt{n}$ and recurse within these blocks. This turns our original search problem of size n into…

  2 problems of size $\sqrt{n} = n^{1/2}$

  4 problems of size $\sqrt{\sqrt{n}} = n^{1/4}$

  8 problems of size $\sqrt{\sqrt{\sqrt{n}}} = n^{1/8}$



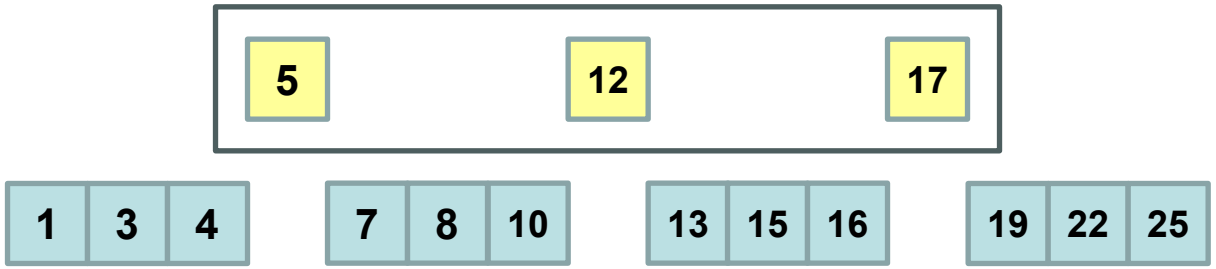
Cache-Oblivious Searching

- Consider the problem of searching a sorted length-n array in the comparison model.
- Optimal cache-aware running time: $O(\log_B n)$ using a B-tree.
- Divide into $\sqrt{n}$ blocks of size $\sqrt{n}$ and recurse within these blocks. This turns our original search problem of size n into…

2 problems of size $\sqrt{n} = n^{1/2}$

4 problems of size $\sqrt{\sqrt{n}} = n^{1/4}$

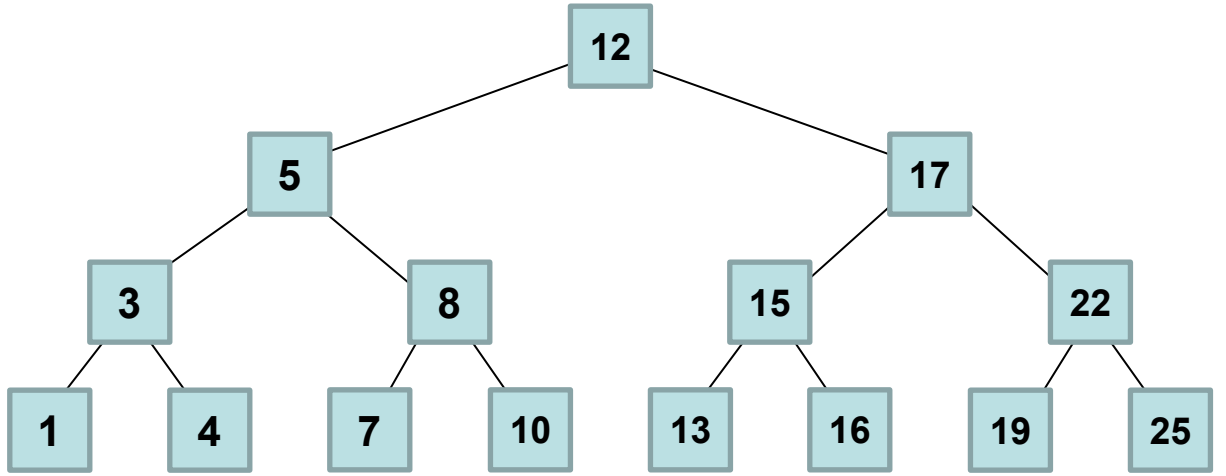
8 problems of size $\sqrt{\sqrt{\sqrt{n}}} = n^{1/8}$



? problems of size $n^{1/?} = B$ → $n^{1/?} = B$ solves to $? = \log_B n$

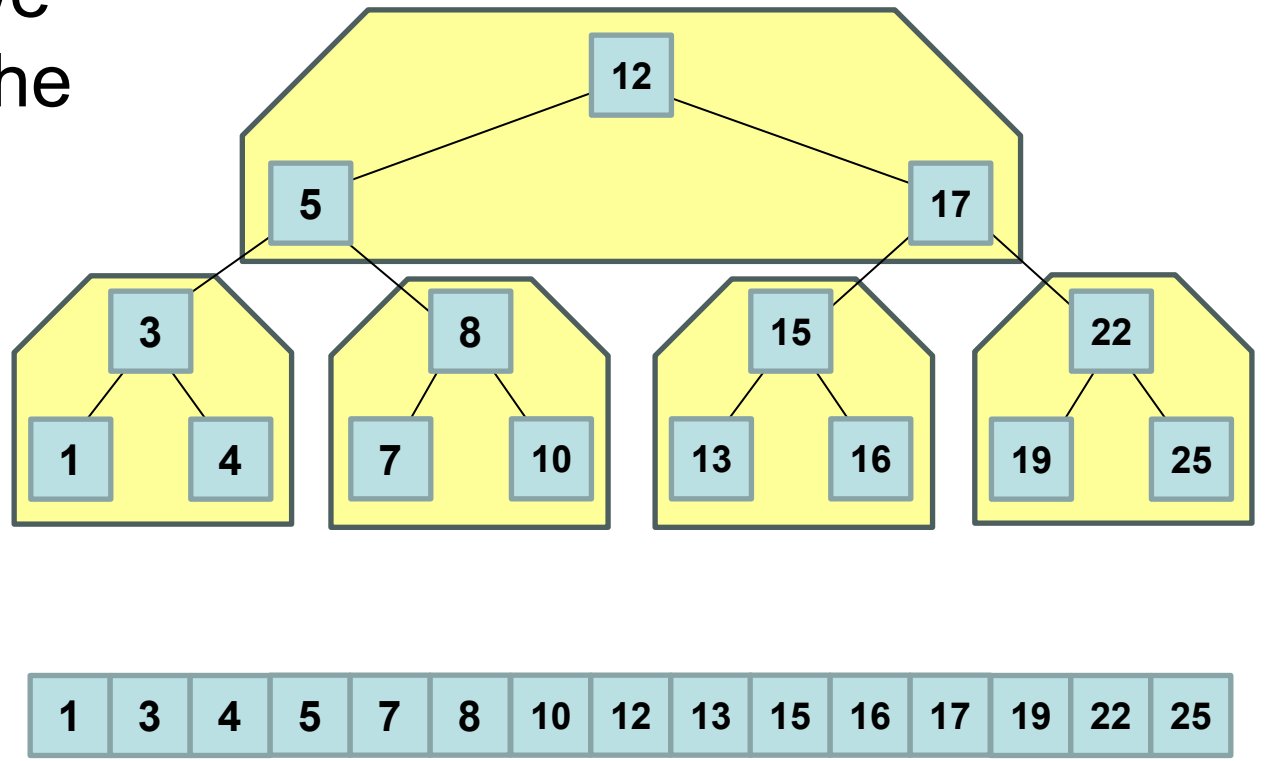
Cache-Oblivious Searching

- Another perspective: build a binary search tree…



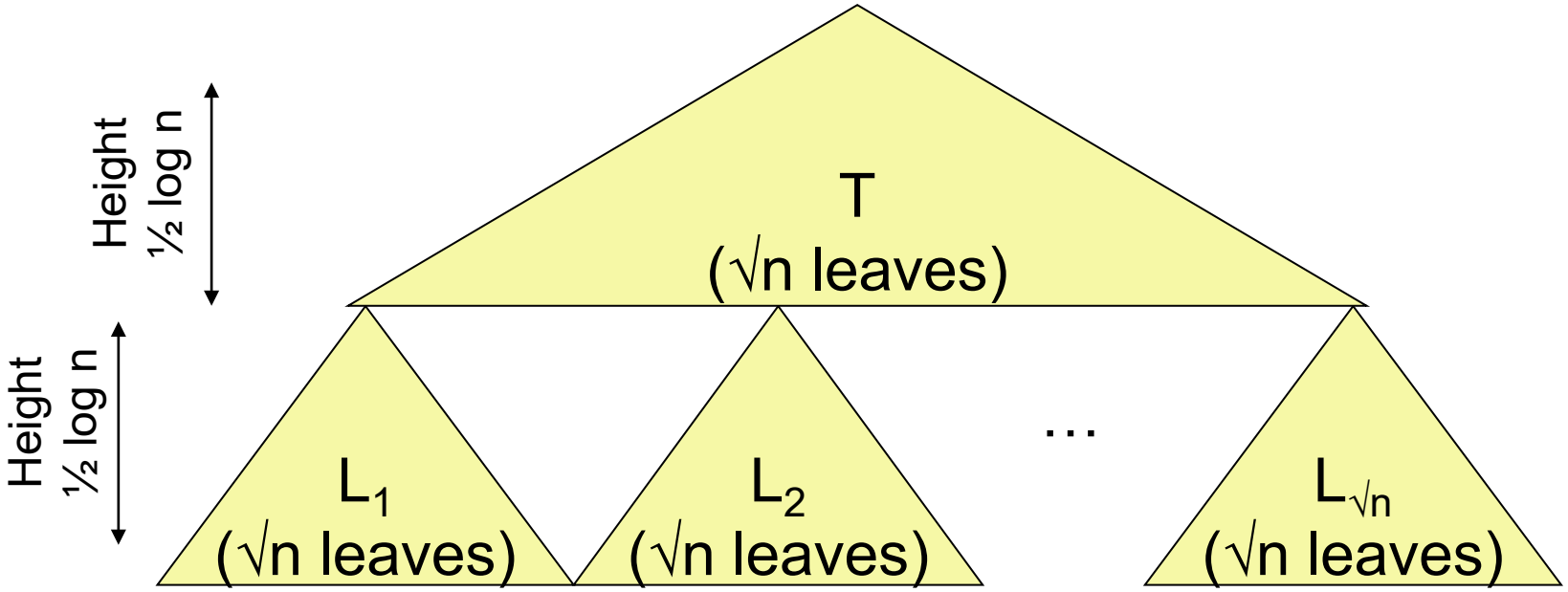
Cache-Oblivious Searching

- Another perspective: build a binary search tree…
- A similar picture emerges if we think of aggregate blocks of the tree as “super-nodes”.



The van Emde Boas (vEB) Tree Layout

- A binary tree of height log n has $2^{\log n} = n$ leaves.
- A binary tree of height ½ log n has $2^{\frac{1}{2}\log n} = \sqrt{n}$ leaves.



- In memory, store T followed by $L_1 \ldots L_{\sqrt{n}}$ (each also recursively subdivided in the same fashion).

Cache-Oblivious Searching with the vEB Tree Layout

- Recursion effectively stops once we reach subtrees that fit into a single block (with O(B) leaves / elements, and height Θ(log B)).
- So a root-leaf path passes through only Θ(log n / log B) = Θ($\log_B$ n) of them (therefore we hit Θ($\log_B$ n) blocks)